



Movie App

A Full-Stack Application (Vue 3 + Express.js + MongoDB)

Semester Project Deliverable 2

Express.js & MongoDB Integration

Student Name:	Umair Azhar / Abdul Aziz
Roll No :	23010101-044 / 23010101-023
Course:	Web Engineering
Instructor:	Sir. Museb
Date:	16 July, 2026

1. Introduction

In Deliverable 1 our Movie App was frontend only. The movies were kept in a file inside the project (src/data/movies.js) and the watchlist was saved in the browser's localStorage. So the data was stuck on one computer, and it was lost if the browser storage was cleared.

In Deliverable 2 we turned the same app into a full-stack application. We built a backend with Express.js and connected it to a MongoDB Atlas database. The movies and the watchlist are now stored in the database not in the frontend. The Vue app keeps no data of its own it asks the backend for everything using Axios and the backend reads and writes that data in MongoDB. We also added basic CSS so the app looks cleaner.

1.1 Technologies added

Tool	Why we used it
Node.js	Lets us run JavaScript on the server instead of in the browser.
Express.js	The framework that listens for requests and sends back responses (our API).
MongoDB Atlas	The cloud database where the movies and watchlist are stored.
Mongoose	Connects Node to MongoDB and lets us define our schema.
Axios	Used by the Vue app to send HTTP requests to our backend.
cors	Lets the frontend (port 5173) call the backend (port 5000).
dotenv	Reads the database connection string from a .env file, so no password is in the code.

Vue 3, Vite, Vue Router and Pinia from Deliverable 1 are all still used. We did not remove anything we added the backend underneath.

1.2 What changed from Deliverable 1

Feature	Deliverable 1	Deliverable 2
Movie data	Local file src/data/movies.js	movies collection in MongoDB
Genre filter	Filtered in the browser	Filtered by the server (?genre=)
Search	Filtered in the browser	Searched by the server (?search=)
Watchlist	Browser localStorage	watchlist collection in MongoDB
Data source	Bundled inside the app	Fetches over HTTP with Axios
Styling	Plain HTML, no CSS	A global stylesheet (src/style.css)

2. Project Structure (Client / Server)

Both parts are in the same GitHub repository but in two separate folders. The client folder is our Vue frontend and the server folder is the new Express backend. Each has its own package.json, so they run independently.

```

Vue-Project/
├── client/
│   └── src/
│       ├── components/
│       ├── pages/
│       ├── router/
│       ├── services/api.js
│       ├── stores/
│       ├── style.css
│       ├── App.vue
│       └── main.js
└── server/
    ├── config/db.js
    └── models/
        ├── movie.js
        └── watchlist.js

```

the Vue 3 frontend

Navbar.vue, MovieCard.vue

Home, Search, MovieDetail, Watchlist

the routes

NEW: all the Axios calls

Pinia store (now talks to the API)

NEW: the global CSS

NEW: the Express + MongoDB backend

opens the MongoDB connection

Movie.js, Watchlist.js (the schemas)

— controllers/	the logic for each endpoint
— routes/	movies.js, watchlist.js
— seed/seedMovies.js	fills the database with the 12 movies
— .env	connection string (NOT on GitHub)
— server.js	starting point of the backend

2.1 Why the backend is split into folders

We did not write the backend in one long file. Each folder has one job the same idea as the components in Deliverable 1.

File / folder	Its job
server.js	Sets up the middleware, connects the routes and starts the server.
config/db.js	Opens the connection to MongoDB Atlas.
models/	Defines the schema, what a document looks like.
controllers/	The logic: read the request, talk to the database, send the response.
routes/	Matches a URL and method (GET / POST / DELETE) to a controller function.
seed/	A one-time script that fills the database with the starting movies.

3. Backend Architecture (Express.js)

The backend is an Express app that gives us a REST API. It connects to MongoDB first and only then starts listening so it never runs without a working database.

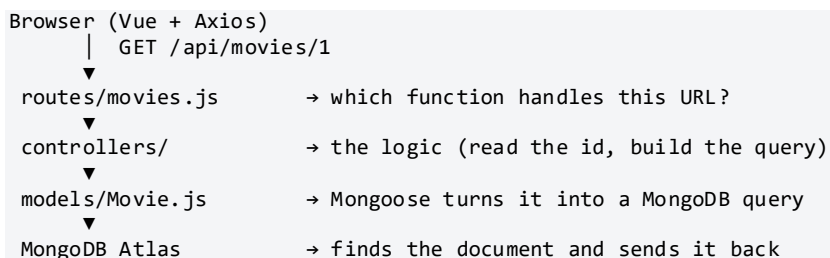
3.1 Middleware

Two middlewares run on every request:

- **cors()** : the frontend is on port 5173 and the backend on port 5000. Browsers block requests between different ports, so this tells the browser our requests are allowed.
- **express.json()** : lets the server read JSON sent in the request body. Without it, req.body would be undefined when we add a movie.

3.2 The request flow

Every request goes through the same four layers. This is the main idea of the whole backend:



3.3 Connecting to the database

The connection is opened once in config/db.js. We never write the connection string in the code, it is read from a .env file through process.env so the password stays out of the source code and off GitHub. A .gitignore file makes sure .env is never pushed.

server/config/db.js

```

export async function connectDB() {
  try {
    await mongoose.connect(process.env.MONGO_URI)
    console.log('Connected to MongoDB Atlas')
  } catch (error) {
    process.exit(1)
  }
}

```

```
}  
}
```

4. Database Design and Schema

Our database is called movieapp. In MongoDB a collection is like a table and each item inside it is called a document. We designed two collections: movies and watchlist.

MongoDB does not force a schema by itself, it would let us save anything into a collection. That is why we use Mongoose. In Mongoose we write a schema which is a description of what fields a document must have and what type each field is. Mongoose then checks every document against it before saving. So our schema lives in the models folder and it is what keeps our data correct and consistent.

4.1 The movies collection

Each document is one movie. We chose the fields to match exactly what our Vue pages already used in Deliverable 1, so we did not have to rewrite any templates.

Field	Type	Explanation
id	Number	Our own simple id (1–12). Required and unique. The frontend uses it in URLs like /movie/1.
title	String	The movie name. Required, because a movie with no title makes no sense.
year	Number	The release year. A Number, not a String, so we could sort or compare by it later.
rating	Number	The rating out of 10. A Number so decimals like 8.4 are stored properly.
runtime	String	The length, e.g. "2h 28m". A String because we only display it, we never do maths with it.
genres	[String]	An array, because one movie can have more than one genre. Defaults to an empty array.
director	String	The director's name. Required.
cast	[String]	An array of actor names, since a movie has many actors.
overview	String	A short description of the story, shown on the detail page.

server/models/Movie.js

```
const movieSchema = new mongoose.Schema({  
  
  id: { type: Number, required: true, unique: true },  
  title: { type: String, required: true },  
  year: { type: Number, required: true },  
  rating: { type: Number, required: true },  
  runtime: { type: String, required: true },  
  genres: { type: [String], default: [] },  
  director: { type: String, required: true },  
  cast: { type: [String], default: [] },  
  overview: { type: String, required: true },  
  
})  
  
export const Movie = mongoose.model('Movie', movieSchema)
```

4.2 The watchlist collection

Each document here is one movie that the user has saved. It only needs two fields.

Field	Type	Explanation
movieId	Number	A reference to a movie's id in the movies collection. Unique, so the same movie cannot be saved twice.
addedAt	Date	When the movie was saved. It fills in the current time automatically, and lets us show the watchlist in the order it was built.

server/models/Watchlist.js

```
const watchlistSchema = new mongoose.Schema({
  movieId: { type: Number, required: true, unique: true },
  addedAt: { type: Date, default: Date.now },
})

export const Watchlist = mongoose.model('Watchlist', watchlistSchema)
```

4.3 Why we designed the schema this way

We store a reference, not a copy. This was our main design decision. The watchlist keeps only the movie's id, not a copy of the whole movie. If we had copied the full movie into the watchlist the same information would be saved in two places. Then if a movie's title was ever corrected the watchlist would still show the old title because it would be holding its own separate copy. By keeping only a reference every movie is stored exactly once. When the watchlist is requested the server takes the saved ids and looks the real movies up in the movies collection using MongoDB's \$in operator.

We kept our own numeric id. MongoDB automatically gives every document a long unique _id like 507f1f77bcf86cd799439011. We kept our own simple id field as well because our routes from Deliverable 1 already use short URLs like /movie/1. Because of this, our router and our links did not have to change at all.

We marked the key fields unique. Both id in movies and movieId in watchlist are unique. This means the database itself blocks duplicates instead of us only trusting our own code to check. It is a safety net at the lowest level.

We used arrays where the data is naturally a list. genres and cast are arrays of strings. This is one advantage of MongoDB in a normal SQL database we would have needed extra tables and joins to store many genres for one movie but here the list simply lives inside the movie document.

4.4 Filling the database (seeding)

A new database starts empty so we wrote a small script (seed/seedMovies.js) to put our original 12 movies into the movies collection. It deletes everything first and then inserts the full list so running it again just resets the data instead of making duplicates. We run it once with npm run seed.

server/seed/seedMovies.js

```
await Movie.deleteMany({})
await Movie.insertMany(movies)
```

5. REST API Endpoints

Our backend has nine endpoints. They follow REST naming: the URL says which resource we want and the method says what to do with it (GET to read, POST to create, DELETE to remove).

Method	Endpoint	What it does
GET	/	Health check shows the API is running.
GET	/api/movies	Returns all movies, sorted by id.
GET	/api/movies?genre=Action	Returns only the movies in that genre.
GET	/api/movies?search=matrix	Returns movies whose title contains that text.
GET	/api/movies/:id	Returns one movie, or 404 if it does not exist.
GET	/api/genres	Returns the list of genres for the dropdown.
GET	/api/watchlist	Returns the full movies the user saved.
POST	/api/watchlist	Adds a movie. Body: { "movieId": 1 }
DELETE	/api/watchlist/:movieId	Removes one movie from the watchlist.
DELETE	/api/watchlist	Clears the whole watchlist.

5.1 Filtering and searching on the server

In Deliverable 1 the genre filter and the search ran in the browser because all the movies were already loaded there. Now the database does that work. The controller reads the query string and builds a MongoDB filter from it. For the search we use a regular expression with the "i" option, which makes it case-insensitive so typing "matrix" still finds "The Matrix".

server/controllers/movieController.js

```
export async function getMovies(req, res) {
  const { genre, search } = req.query
  const filter = {}

  if (genre && genre !== 'All') {
    filter.genres = genre
  }
  if (search) {
    filter.title = { $regex: search, $options: 'i' } // 'i' = case-insensitive
  }

  const movies = await Movie.find(filter).sort({ id: 1 })
  res.json(movies)
}
```

This is better than before because only the movies we actually need travel over the network, instead of sending all of them and filtering in the browser.

5.2 Status codes and error handling

Every controller is wrapped in a try / catch, so a database problem sends back a clear JSON error instead of crashing the server. We use the standard HTTP codes:

- **200 OK** : the request worked.
- **201 Created** : a movie was added to the watchlist.
- **400 Bad Request** : the request was missing the movieId.
- **404 Not Found** : no movie exists with that id.
- **500 Internal Server Error** : something went wrong on the server or database.

5.3 Getting the watchlist

Because the watchlist only stores ids, this controller does two steps: first it reads the saved entries then it uses those ids to find the real movies with the \$in operator, which matches any document whose id is in the list we give it.

server/controllers/watchlistController.js

```
const entries = await Watchlist.find().sort({ addedAt: 1 })
const ids = entries.map((entry) => entry.movieId)
const movies = await Movie.find({ id: { $in: ids } })
res.json(movies)
```

6. Connecting the Frontend to the Backend (Axios)

Axios is a JavaScript library for sending HTTP requests. Our Vue app uses it to call the backend whenever a page needs data. We chose Axios because it is short to write, it turns the JSON response into a JavaScript object automatically and it treats error responses like 404 as real errors we can catch.

6.1 One central API file

Instead of writing Axios calls inside every page, we put them all in one file: `src/services/api.js`. We create an Axios instance with a `baseUrl` so the backend address is written in only one place. If the backend ever moves we only change that one line.

client/src/services/api.js

```
import axios from 'axios'

const api = axios.create({
  baseUrl: 'http://localhost:5000/api',
})

export async function getMovies(genre) {
  const res = await api.get('/movies', { params: { genre } })
  return res.data
}

export async function getMovieById(id) {
  try {
    const res = await api.get('/movies/' + id)
    return res.data
  } catch (error) {
    return null
  }
}

export async function addToWatchlist(movieId) {
  await api.post('/watchlist', { movieId })
}

export async function removeFromWatchlist(movieId) {
  await api.delete('/watchlist/' + movieId)
}
```

We use the `params` option instead of joining the text onto the URL ourselves. Axios builds the query string for us, so `/movies` with `params { genre: "Sci-Fi" }` becomes `/api/movies?genre=Sci-Fi` with the special characters encoded correctly.

6.2 Which page calls which endpoint

Page / file	Function used	Endpoint it calls
HomePage.vue	getMovies(genre)	GET /api/movies?genre=
HomePage.vue	getGenres()	GET /api/genres
SearchPage.vue	searchMovies(word)	GET /api/movies?search=
MovieDetailPage.vue	getMovieById(id)	GET /api/movies/:id
App.vue (on load)	watchlist.load()	GET /api/watchlist
stores/watchlist.js	add() / remove()	POST / DELETE /api/watchlist
WatchlistPage.vue	watchlist.clear()	DELETE /api/watchlist

6.3 The Pinia store now uses the database

In Deliverable 1 the store saved to localStorage. Now each action first tells the backend what to do then updates the local list so the screen stays in sync. From the pages point of view nothing changed they still call watchlist.toggle(movie) exactly as before, which is why our components did not need rewriting.

client/src/stores/watchlist.js

```
actions: {
  async load() {
    this.movies = await getWatchlist()
  },
  async add(movie) {
    await addToWatchlist(movie.id)
    this.movies.push(movie)
  },
  async remove(id) {
    await removeFromWatchlist(id)
    this.movies = this.movies.filter((m) => m.id !== id)
  },
  async toggle(movie) {
    if (this.isSaved(movie.id)) {
      await this.remove(movie.id)
    } else {
      await this.add(movie)
    }
  },
}
```

6.4 Loading data when a page opens

The data now comes over the network so it is not there instantly. Our pages use onMounted() to fetch as soon as they open and watch() to fetch again when something changes for example when the user picks a new genre. The saved watchlist is loaded once for the whole app in App.vue, so the Navbar count is correct no matter which page opens first.

client/src/pages/HomePage.vue

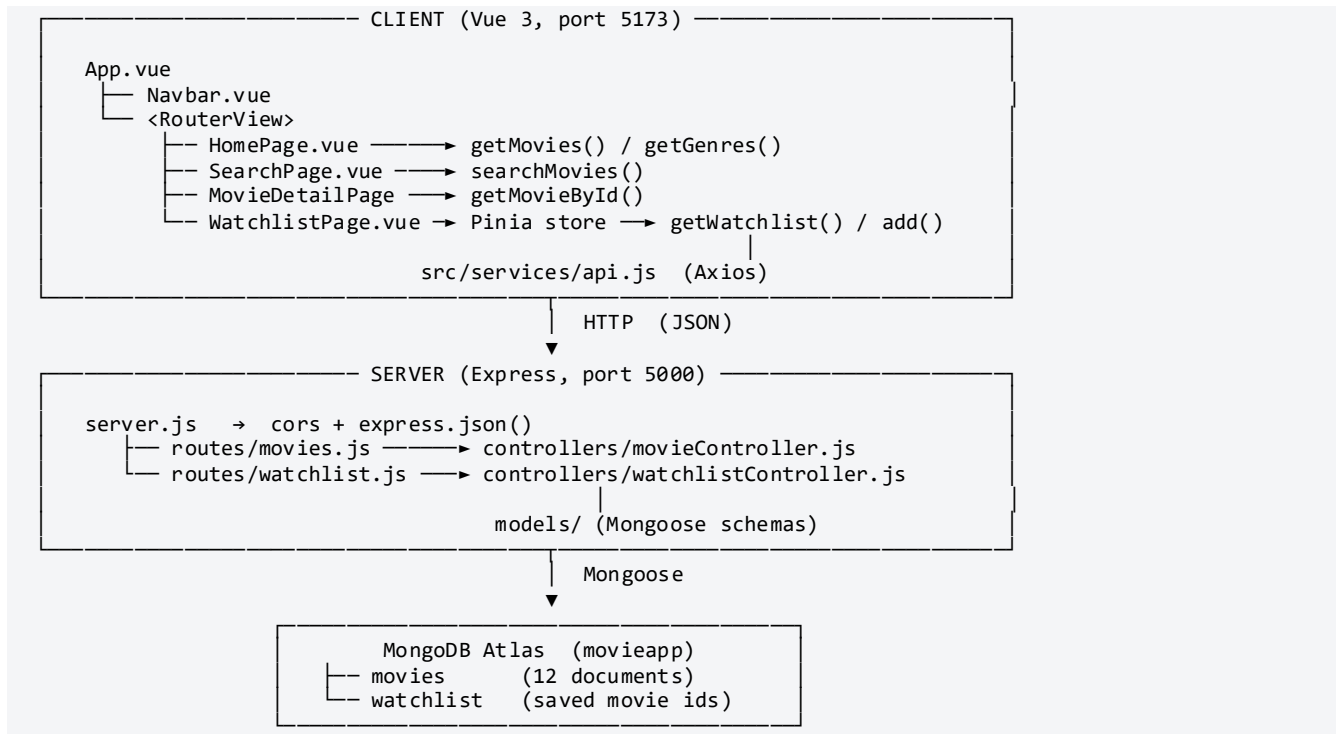
```
const movies = ref([])
const allGenres = ref([])

async function loadMovies() {
  movies.value = await getMovies(selectedGenre.value)
}

onMounted(async () => {
  allGenres.value = await getGenres()
  await loadMovies()
}) watch(selectedGenre, loadMovies)
```


7. Full-Stack Architecture Diagram

The diagram shows the whole application. The top part is our Vue frontend from Deliverable 1, everything from Axios down is new. Data flows in one direction: the components ask the store or the API file, which asks Express, which asks MongoDB.



7.1 A full example: adding a movie to the watchlist

Following one click through every layer shows how the whole stack fits together:

- The user clicks "Add to watchlist" on a MovieCard.
- The card emits the toggle event up to its parent page (exactly as in Deliverable 1).
- The page calls `watchlist.toggle(movie)` on the Pinia store.
- The store calls `addToWatchlist(movie.id)` in `api.js`.
- `Axios` sends `POST http://localhost:5000/api/watchlist` with the body `{ "movieId": 1 }`.
- `Express` matches the route in `routes/watchlist.js` and calls the controller.
- The controller checks the movie exists and is not already saved, then creates the document with `Mongoose`.
- `MongoDB` stores it, the server replies `201 Created` and the store adds the movie to its list so the `Navbar` count updates instantly.

8. Conclusion

In this deliverable we changed the Movie App from a frontend-only project into a complete full-stack application. We built a REST API with `Express.js`, designed a `MongoDB` database with two collections and connected our Vue frontend to that API using `Axios`. The biggest improvement is where the data lives. In Deliverable 1 the movies were fixed inside the code and the watchlist existed in only one browser. Now both are in a real database, so the data is permanent, it can be changed without touching the frontend code and it would be available to any user instead of just one browser.